COURSE OUTLINE

Gennoor Tech
TRAIN. INNOVATE. BUILD.

Terraform and GitHub Automation with Claude

Terraform step by step, the mistakes that burn teams, and a GitHub workflow where Claude reviews and a human approves

3 days (8 hours per day) | Beginner to intermediate | Instructor-led, hands-on labs | Capstone project

About this course

Who should attend

Enterprise IT and network operations teams, load balancer and data center engineers, platform and DevOps teams adopting infrastructure as code.

Prerequisites

- Basic Linux command line and text editing
- Basic networking and load balancer concepts (VIP, pool, certificate)
- Git basics are helpful, not required
- No prior Terraform or Python experience required

What you will be able to do

- Write, plan and apply Terraform configurations step by step with providers, variables, modules, remote state and locking
- Recognise and fix the common Terraform mistakes: unpinned providers, secrets in code, state in Git, count versus for_each, targeted applies and drift
- Run a GitHub repository for infrastructure code with branches, pull requests, CODEOWNERS, branch protection and GitHub Actions plan-on-PR and apply-on-merge
- Set up Claude Code, the Claude Agent SDK and an MCP server so Claude can draft, plan and review Terraform without ever running apply itself
- Build a plan-review agent that reads terraform show -json output and posts a structured risk verdict to the pull request
- Enforce guardrails, environment approvals and an audit trail for every plan and apply across F5, NetScaler, Cisco ACI and other load balancers (Anaya LB)

Gennoor Tech TRAIN. INNOVATE. BUILD. For more info - https://gennoor.com

Tools and technologies

Terraform CLI | HashiCorp providers | GitHub | GitHub Actions | Claude Code and Agent SDK | Model Context Protocol (MCP) | Python | F5 BIG-IP provider | Citrix ADC provider | Cisco ACI provider | Radware Alteon REST | Linux

Table of contents

DAY 1 3 modules

MODULE 1

Infrastructure as code for network and load balancer teams

- Why Terraform for F5, NetScaler, Cisco ACI and cloud: declarative config versus click-ops and ad hoc scripts
- HCL building blocks: providers, resources, data sources, variables, outputs and the state file
- The course pattern: Claude plans and explains, Terraform and pipelines execute, a human approves, everything is logged
- Lab environment tour: GitHub organization, lab F5 and NetScaler, Cisco ACI simulator, Terraform CLI on Linux

HANDS-ON LAB
Install Terraform, clone the course repository and run terraform init and plan against a sample F5 BIG-IP pool configuration.

DELIVERABLE
Working Terraform workstation and first successful plan.

Genpoor Tech PRIVATE. BUILD. For more info - https://genpoor.com

MODULE 2

Setting up Claude Code, the Agent SDK and MCP for Terraform work

- Claude Code in the repository: CLAUDE.md conventions, allowed commands, and why apply is never on the allowed list
- Claude Agent SDK basics in Python: a session, tool use, structured JSON output with a schema
- An MCP server that exposes terraform fmt, validate, plan and show -json as read-only tools
- Prompting Claude to explain a plan in plain English and cite the resource addresses it refers to

HANDS-ON LAB
Configure Claude Code for the repo, start the Terraform MCP server, and ask Claude to explain a plan without being able to apply it.

DELIVERABLE
Claude Code plus MCP setup with a read-only Terraform tool set.

MODULE 3

Terraform scripting step by step: init, plan, apply, state

- The lifecycle: init, fmt, validate, plan, apply, destroy; reading a plan symbol by symbol (+, -, ~, -/+)
- Variables, tfvars files, locals, outputs and the first reusable module for an F5 virtual server and pool
- terraform import for a virtual server that already exists; terraform state list, show, mv and rm
- Remote state with locking, workspaces for dev and prod, and pinned provider versions in required_providers

HANDS-ON LAB
Build a module for an F5 virtual server with Claude drafting the HCL, import an existing pool, and move state to a locked remote backend.

DELIVERABLE
Reusable virtual server module with remote, locked state.

DAY 2 4 modules

GenNoorTech TRAIN · INNOVATE · BUILD
For more info - https://gennoor.com

MODULE 4

Common Terraform mistakes and how to catch them early

- Unpinned provider versions, secrets hard coded in variables, and the .tfstate file committed to Git
- count versus for_each, resource recreation from a renamed block, and the moved block as the fix
- Drift left behind by terraform apply -target, manual changes on the device, and detecting drift with plan -detailed-exitcode
- Wiring the catchers: terraform fmt -check, validate, tflint, a pre-commit hook and a Claude review that names the mistake and the line

HANDS-ON LAB
Reproduce five classic mistakes in the lab repo, watch each one surface in plan or review, and fix them one by one.

DELIVERABLE
Mistake catalogue with reproduction and fix for each, plus pre-commit checks.

MODULE 5

GitHub repository fundamentals for infrastructure code

- Repository layout for Terraform: environments, modules, .gitignore for state, plans and credentials
- Branches, commits, pull requests, reviews and merge strategies for a network team
- CODEOWNERS, branch protection rules, required reviewers and status checks
- Secrets and variables in GitHub: repository secrets, environment secrets, OIDC to cloud and Vault, and short-lived device credentials fetched at run time

HANDS-ON LAB
Create the team repository, protect main, add CODEOWNERS for the F5 and ACI folders, and raise your first pull request from a feature branch.

DELIVERABLE
Protected GitHub repository with pull request workflow.

Gelnoor Innovate Build
For more info - https://gelnoor.com

MODULE 6

GitHub Actions pipelines: plan on pull request, apply on merge

- Workflow anatomy: triggers, jobs, hashicorp/setup-terraform, caching and concurrency groups to avoid parallel applies
- Plan on every pull request with the plan output posted as a PR comment; fmt, validate and tflint as required checks
- Apply only on merge to main, scoped to a GitHub environment with required reviewers
- Saving the plan file as an artifact so the apply uses exactly the plan that was reviewed

HANDS-ON LAB
Write the plan and apply workflows with Claude Code, open a pull request, read the plan comment and merge to see the gated apply run.

DELIVERABLE
Working plan-on-PR and apply-on-merge pipelines.

MODULE 7

Guardrails, approvals and audit for Terraform and GitHub automation

- Policy checks before apply: allowed resource types, no destroy without a linked change ticket, no changes outside the PR scope
- GitHub environment protection rules and required reviewers as the human approval gate for every apply
- Audit record per change: who asked, what Claude recommended, plan hash, approver, apply result and run URL
- Least privilege for pipelines and agents: read-only device accounts for plan, scoped tokens, no long-lived credentials in the pipeline

HANDS-ON LAB
Add a policy job that blocks a destroy without a ticket, require approval on the prod environment, and export the audit record for one change.

DELIVERABLE
Guardrail policy, approval gate and audit record format.

DAY 3 3 modules

genoor tech
TRAIN. INNOVATE. BUILD.
For more info-https://genoor.com

MODULE 8

Claude as plan reviewer: Agent SDK, MCP and structured verdicts

- An Agent SDK application that reads terraform show -json and returns risk level, affected resources and a summary in a fixed JSON schema
- Posting the verdict as a pull request comment and as a check run through the GitHub API; the agent never merges or applies
- Prompt and schema tests: deletes, replacements, certificate changes and out-of-scope resources must be flagged correctly
- Handling uncertainty: the agent marks unknowns and asks for a human when the plan touches resources it cannot classify

HANDS-ON LAB

Build and test the plan-review agent, run it inside the pull request workflow and watch it flag a hidden resource replacement.

DELIVERABLE

Plan-review agent with schema tests wired into the PR pipeline.

MODULE 9

Terraform for F5, NetScaler, Cisco ACI and other load balancers

- F5 BIG-IP provider: bigip_ltm_node, bigip_ltm_pool, bigip_ltm_virtual_server and bigip_ssl_certificate backed by iControl REST
- Citrix ADC provider for NetScaler: citrixadc_lbvserver, citrixadc_servicegroup and bindings over the NITRO API
- Cisco ACI provider: aci_tenant, aci_application_profile, aci_bridge_domain against the APIC REST API
- Where no provider exists: Python wrappers for Radware Alteon and other load balancers (Anaya LB) driven from the same pipeline with the same approval gate

HANDS-ON LAB

Add a NetScaler virtual server and an ACI bridge domain through the pull request workflow, and attach a Python-driven Radware change to the same gate.

DELIVERABLE

Multi-platform Terraform modules behind one approval workflow.

GenNoorTech
For more info - https://gennoor.com

MODULE 10

Capstone: end-to-end change through the Claude-reviewed pipeline

- A change request in, a reviewed pull request, a Claude verdict, a human approval, a gated apply and an audit record out
- Runbook: how to raise, review, approve, apply and revert a change in this repository
- Handover: CODEOWNERS, environment reviewers and audit export for the team
- Acceptance review: mistakes caught, approvals enforced, rollback by reverting the pull request, demo to the group

HANDS-ON LAB
Capstone build and demo.

DELIVERABLE
End-to-end Terraform and GitHub workflow with Claude review and runbook.

Capstone

Capstone project

A team GitHub repository where Terraform changes for F5, NetScaler, Cisco ACI and other load balancers (Anaya LB) go through a pull request, a Claude plan review, policy checks and a human approval before a gated apply, with an audit record for every change and rollback by revert.

Assessment

Assessment area	Weight	Evidence
Working functionality	25%	Demonstrable end-to-end workflow in a repeatable lab
Security and safety controls	20%	Threat model, authorization boundaries, approval gates, safe failure
Testing and evaluation	20%	Automated tests, golden dataset, measured quality and known limitations
Engineering quality	15%	Clear interfaces, error handling, reproducible setup, maintainable code
Evidence and explainability	10%	Traceable findings, sources, assumptions, logs
Documentation and presentation	10%	Architecture diagram, README, runbook, demo, lessons learned

For more info - https://gmoor.com

Engineering principles used throughout

- Model output is a proposal, not a fact: every finding and generated change is validated by deterministic checks and tests.
- Authorization lives outside the model: tool access is enforced by application code, scoped credentials and policy.
- Human approval for consequential actions: deploying a mitigation, changing production configuration, rotating credentials.
- Decisions, evidence, model and tool versions, approvals and outcomes are logged without recording secrets.
- A model adapter and a tool adapter layer keep every build provider-agnostic and enterprise-portable.
- Quality is measured: false positives, false negatives, reliability, latency, cost and safe failure, not just a demo.

© Gennoor Tech · Train. Innovate. Build.

For more info - https://gennoor.com

Gennoor Tech
TRAIN · INNOVATE · BUILD
For more info - https://gennoor.com